Heap Binária

Estruturas de Dados

adalberto@computacao.ufcg.edu.br

Motivação

Em BST (não balanceadas) temos tempo das operações sendo Ω(log n) e O(n)

BST possuem sua aplicabilidade

E se quisermos operações de máximo/mínimo em O(1)?

E se a busca não nos interessa?

Heap Binária

Árvore binária satisfazendo as seguintes propriedades/invariantes:

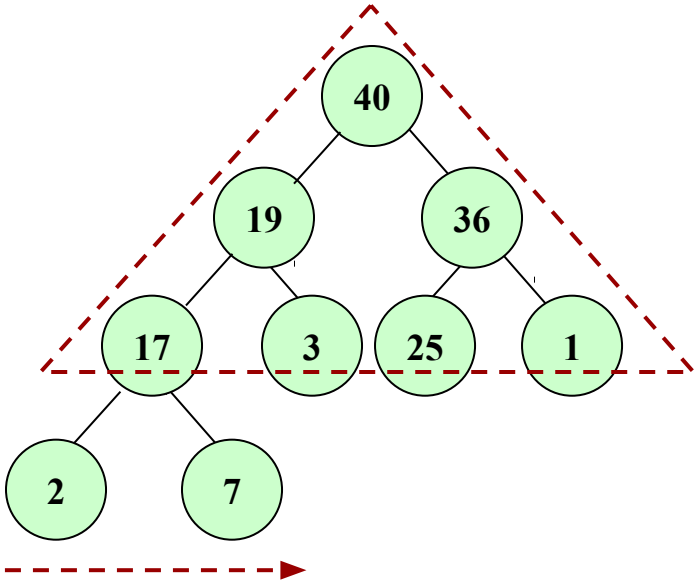
Shape property:

- Estar completa até seu penúltimo nível (pelo menos)
- Ser preenchida de cima para baixo e da esquerda para a direita → se o último nível não estiver completo, deve ser preenchido da esquerda para a direita

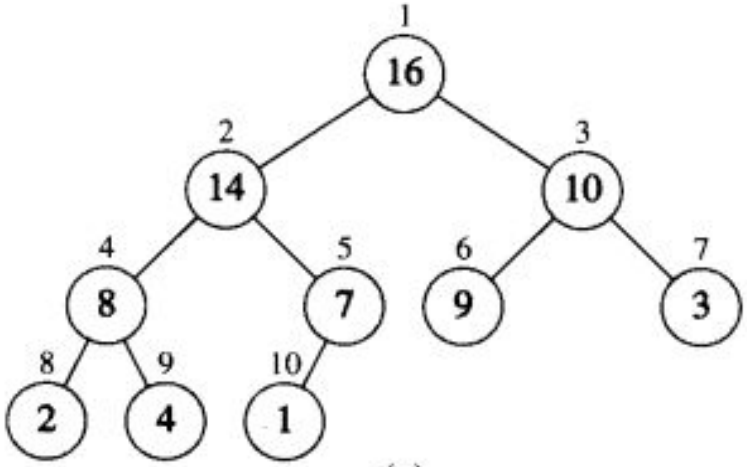
Tipo de heap (max ou min) - ordem:

- Max-heap: todo nó é maior do que seus filhos
- Min-heap: todo nó é menor do que seus filhos

Heap Binária (Exemplo)

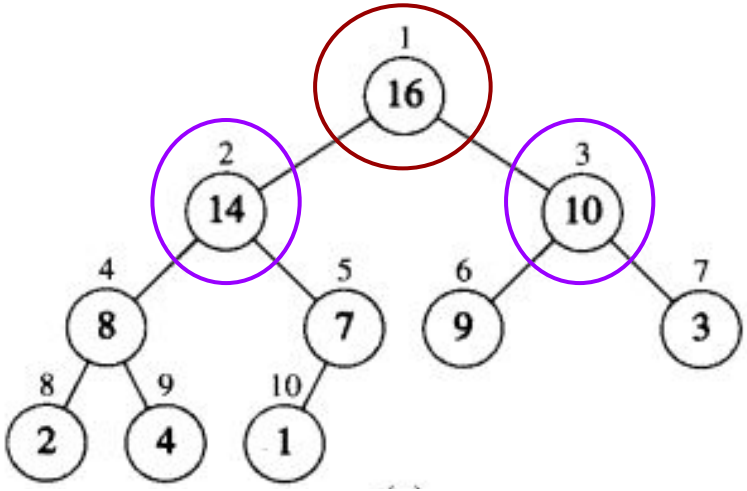


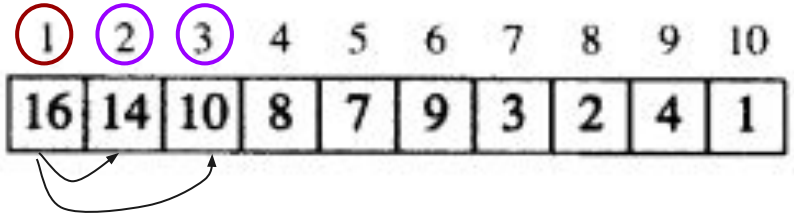
Heap Binária (Representação)



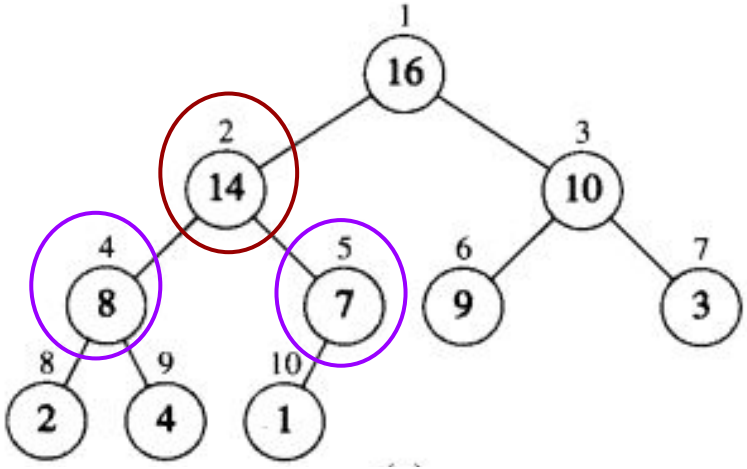
1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

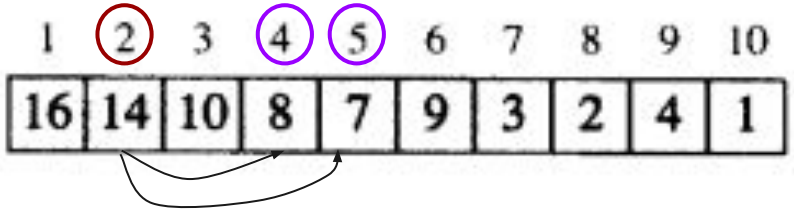
Heap Binária (Representação)



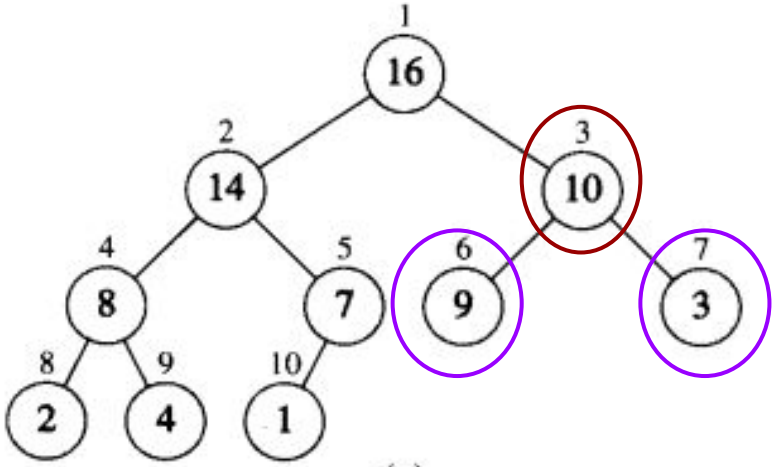


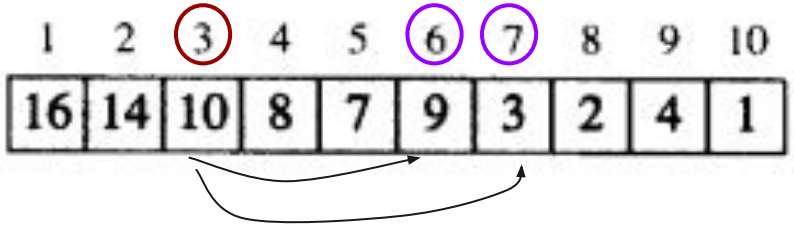
Heap Binária (Representação)



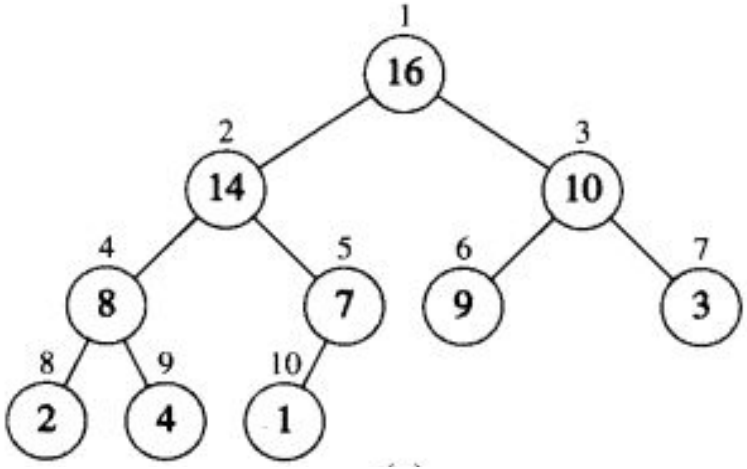


Heap Binária (Representação)





Heap Binária (Representação)



1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

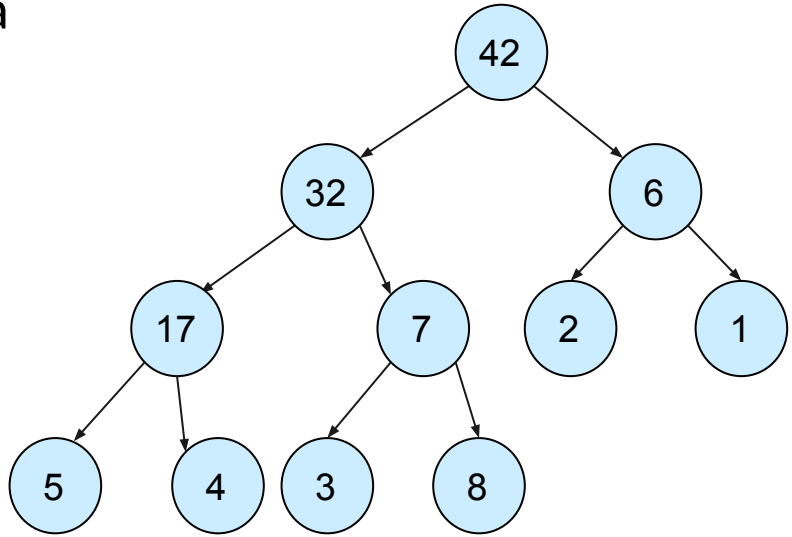
PARENT(i)
return $\lfloor i/2 \rfloor$

RIGHT(i)
return $2i + 1$

LEFT(i)
return $2i$

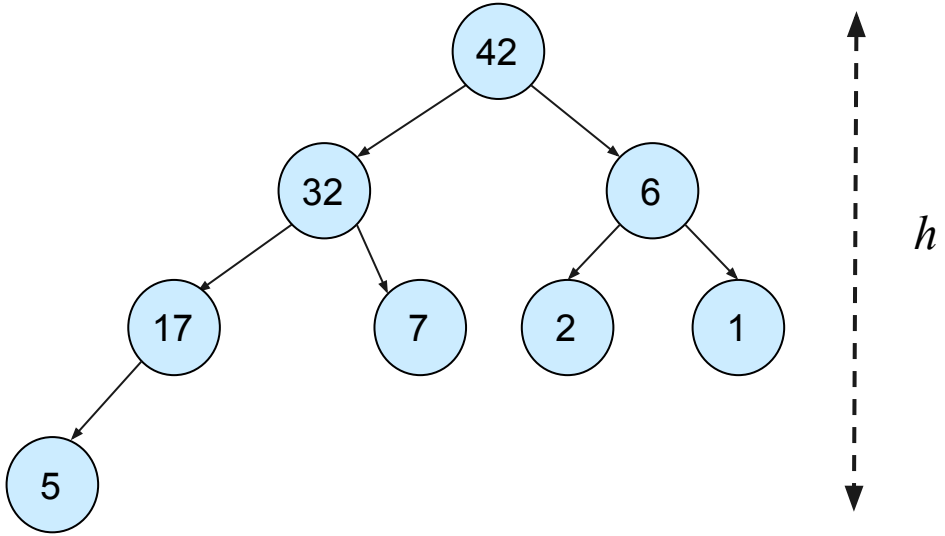
Heap Binária (Exemplo)

A árvore binária representa uma max-heap?



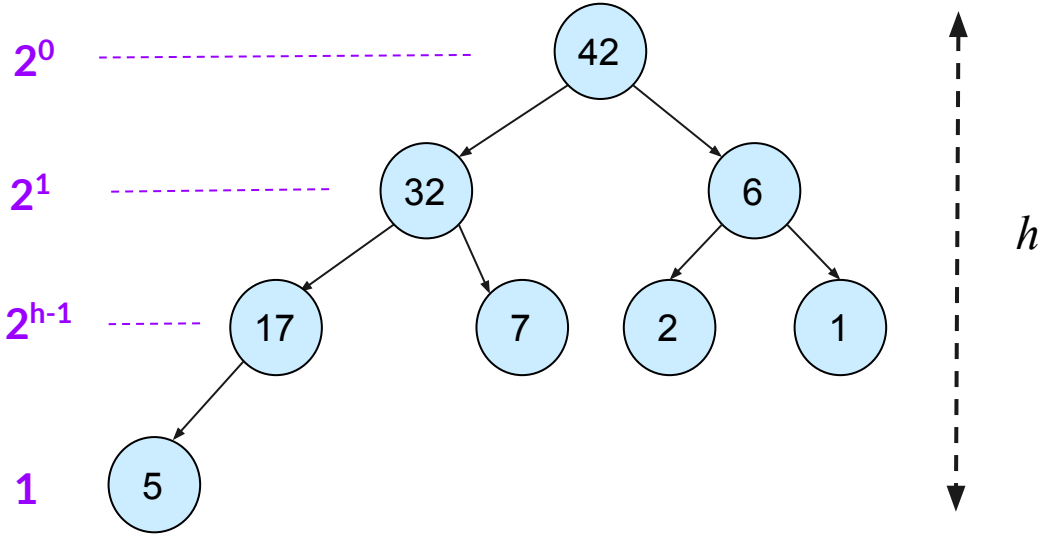
Heap Binária (Propriedades)

Qual o número mínimo de elementos de uma heap com altura h?



Heap Binária (Propriedades)

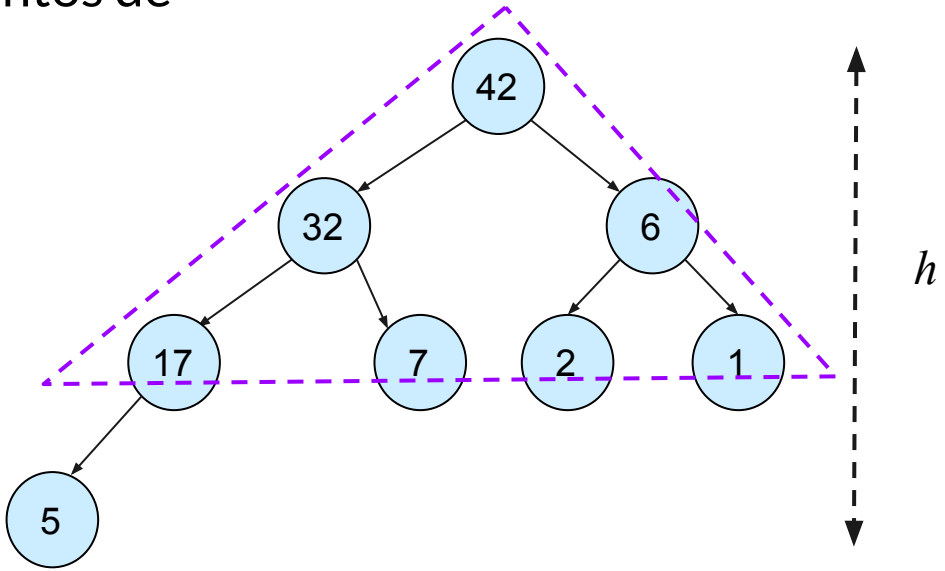
Qual o número mínimo de elementos de uma heap com altura h?



Heap Binária (Propriedades)

Qual o número mínimo de elementos de uma heap com altura h?

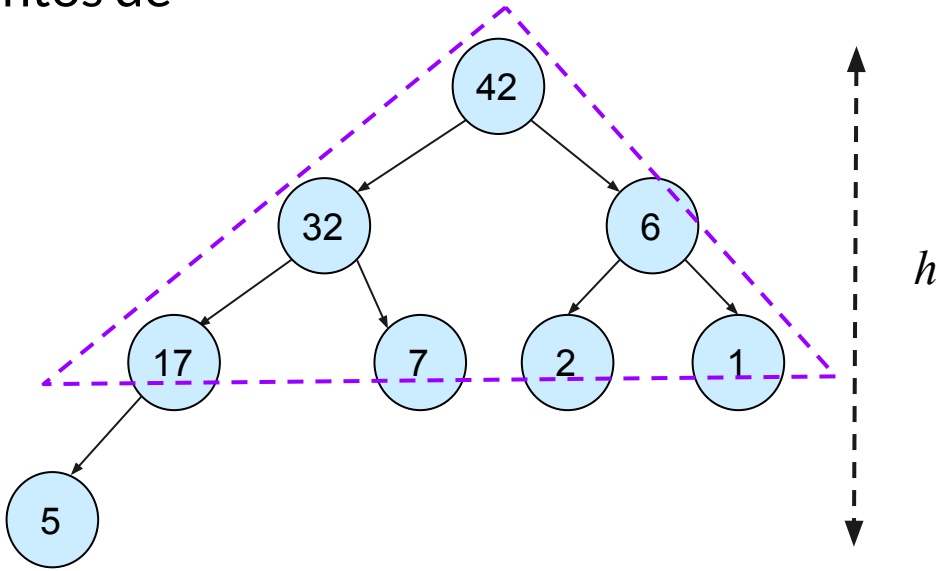
$$S = 2^{h+1} - 1$$



Heap Binária (Propriedades)

Qual o número mínimo de elementos de uma heap com altura h?

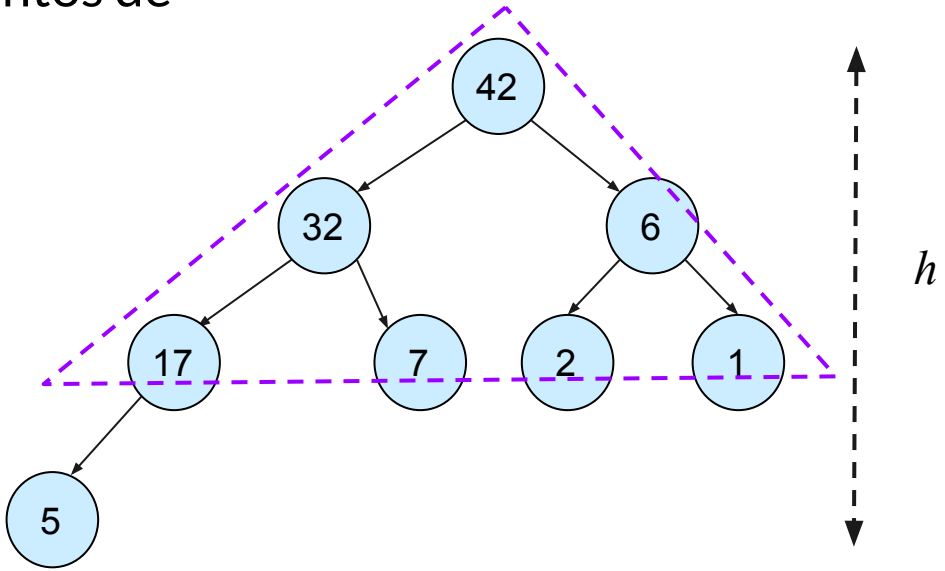
$$S = 2^{h-1+1} - 1$$



Heap Binária (Propriedades)

Qual o número mínimo de elementos de uma heap com altura h?

$$S = 2^h - 1$$



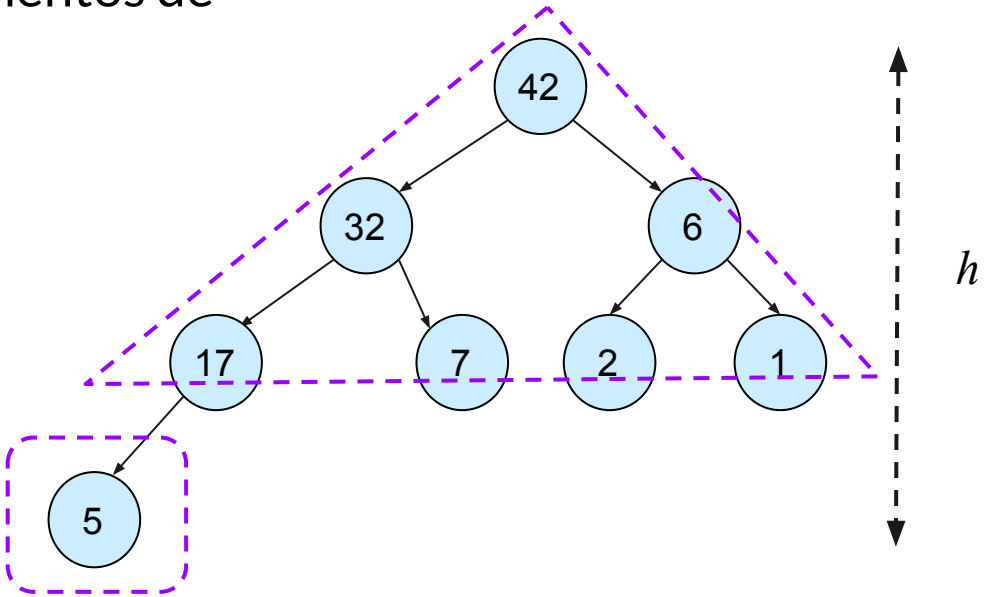
Heap Binária (Propriedades)

Qual o número mínimo de elementos de uma heap com altura h?

$$S = 2^h - 1 + 1$$

$$\Downarrow$$

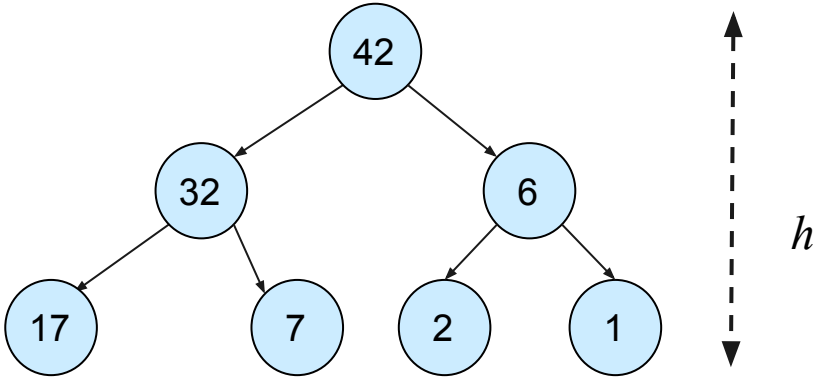
$$2^h$$



Heap Binária (Propriedades)

Qual o número máximo de elementos de uma heap com altura h?

$$S = 2^{h+1} - 1$$



Heap Binária (Propriedades)

Qual a faixa de valores para a altura de uma heap em função do número de elementos?

Heap Binária (Propriedades)

Qual a faixa de valores para a altura de uma heap em função do número de elementos?

$$\textit{heap mínima com altura } h \leq \quad h \quad \leq \quad \textit{heap máxima com altura } h$$

Heap Binária (Propriedades)

Qual a faixa de valores para a altura de uma heap em função do número de elementos?

$$2^h \leq h \leq 2^{h+1} - 1$$

Heap Binária (Propriedades)

Qual a condição necessária e suficiente para descrever o invariante de uma max-heap?

Como manter o invariante de uma max-heap?

Qual o custo das operações sobre uma heap?

Heap Binária (Propriedades)

Qual a condição necessária e suficiente para descrever o invariante de uma max-heap?

$$\forall i \neq 1 \,.\, A[i] \leq A[parent(i)]$$

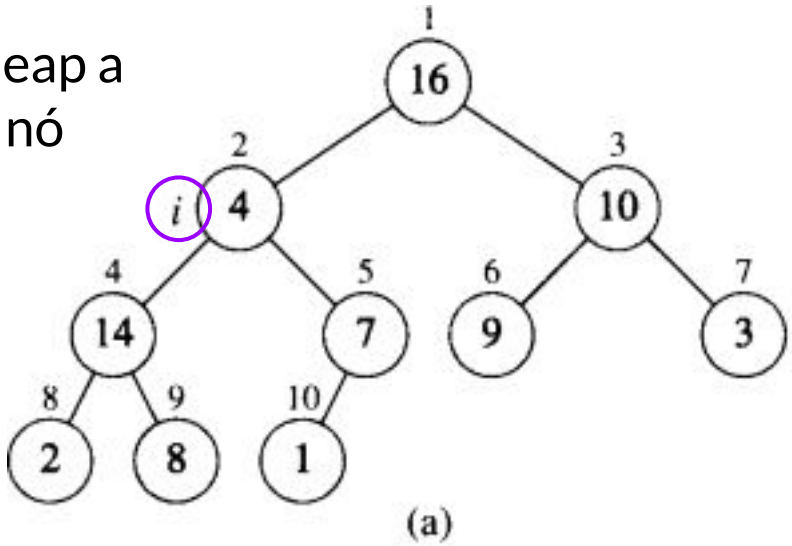
Como manter o invariante de uma max-heap?

Qual o custo das operações sobre uma heap?

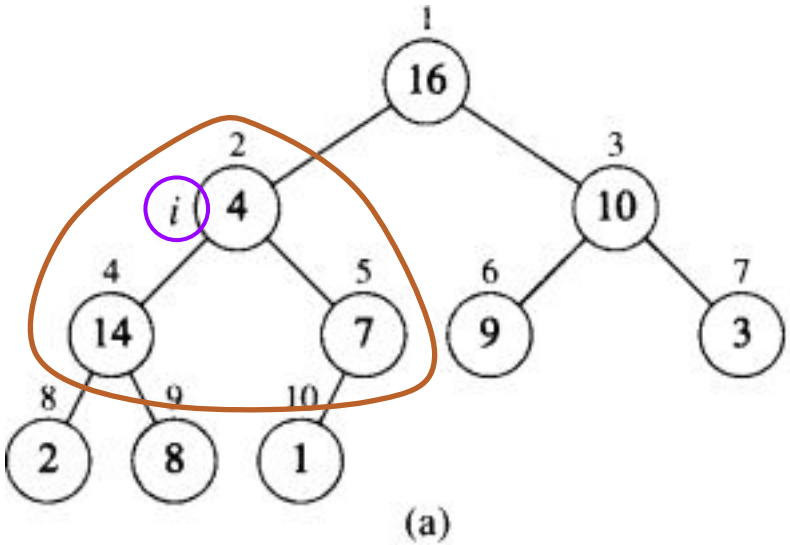
Heap Binária
Algoritmos

Heap Binária (heapify)

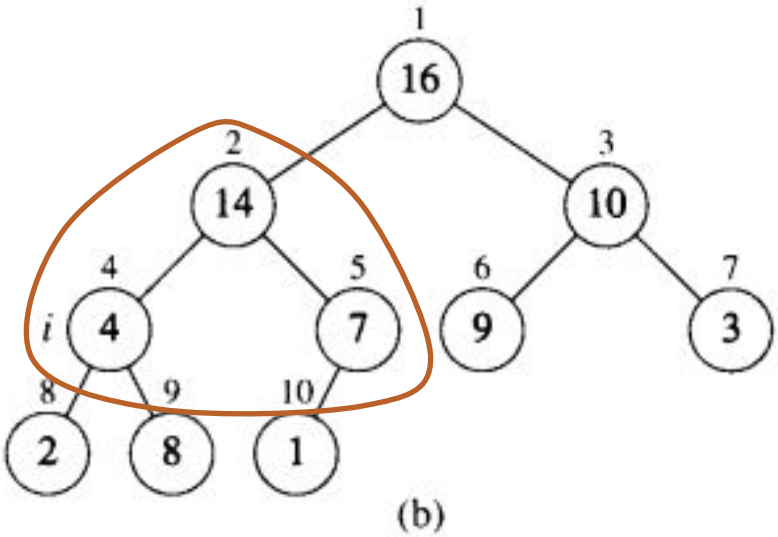
Conserta a heap a partir de um nó



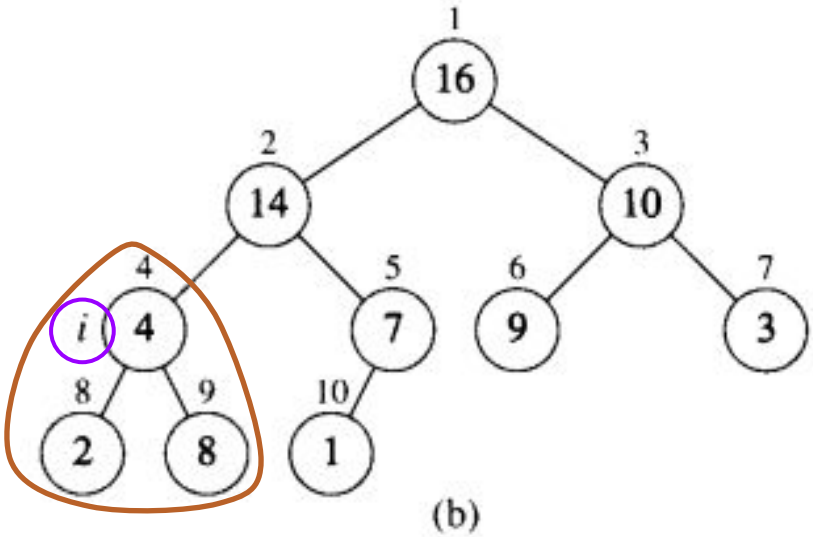
Heap Binária (heapify)



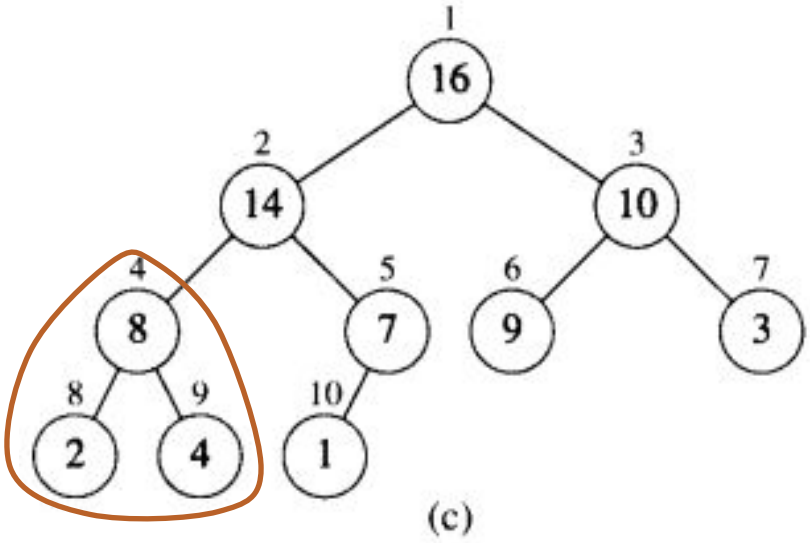
Heap Binária (heapify)



Heap Binária (heapify)



Heap Binária (heapify)



Heap Binária (heapify)

```
HEAPIFY(A, i)
l ← LEFT(i)
r ← RIGHT(i)
if l ≤ heap-size[A] and A[l] > A[i]
    then largest ← l
    else largest ← i
if r ≤ heap-size[A] and A[r] > A[largest]
    then largest ← r
if largest ≠ i
    then exchange A[i] ↔ A[largest]
         HEAPIFY(A, largest)
```

Heap Binária (heapify)

```
HEAPIFY(A, i)
l ← LEFT(i)
r ← RIGHT(i)
if l ≤ heap-size[A] and A[l] > A[i]
    then largest ← l
    else largest ← i
if r ≤ heap-size[A] and A[r] > A[largest]
    then largest ← r
if largest ≠ i
    then exchange A[i] ↔ A[largest]
         HEAPIFY(A, largest)
```

Heap Binária (heapify)

```
HEAPIFY(A, i)
l ← LEFT(i)
r ← RIGHT(i)
if l ≤ heap-size[A] and A[l] > A[i]
    then largest ← l
    else largest ← i
if r ≤ heap-size[A] and A[r] > A[largest]
    then largest ← r
if largest ≠ i
    then exchange A[i] ↔ A[largest]
         HEAPIFY(A, largest)
```

Heap Binária (heapify)

```
HEAPIFY(A, i)
l ← LEFT(i)
r ← RIGHT(i)
if l ≤ heap-size[A] and A[l] > A[i]
    then largest ← l
    else largest ← i
if r ≤ heap-size[A] and A[r] > A[largest]
    then largest ← r
if largest ≠ i
    then exchange A[i] ↔ A[largest]
         HEAPIFY(A, largest)
```

Heap Binária (heapify)

$T(n) = T(n/2) + O(1)$

$\Downarrow$

$O(\log n)$

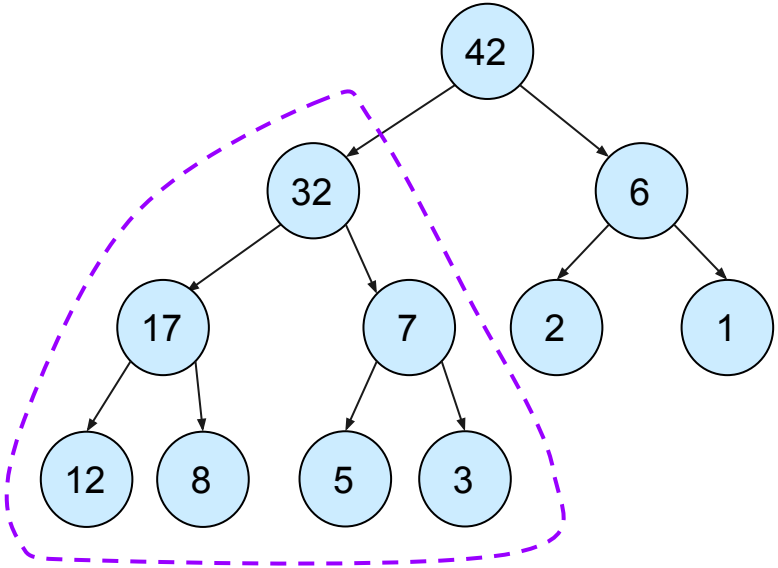
```
HEAPIFY(A, i)
  l ← LEFT(i)
  r ← RIGHT(i)
  if l ≤ heap-size[A] and A[l] > A[i]
      then largest ← l
      else largest ← i
  if r ≤ heap-size[A] and A[r] > A[largest]
      then largest ← r
  if largest ≠ i
      then exchange A[i] ↔ A[largest]
           HEAPIFY(A, largest)
```

Heap Binária (heapify)

$$T(n) = T(2n/3) + O(1)$$

$$\Downarrow$$

$$O(\log n)$$

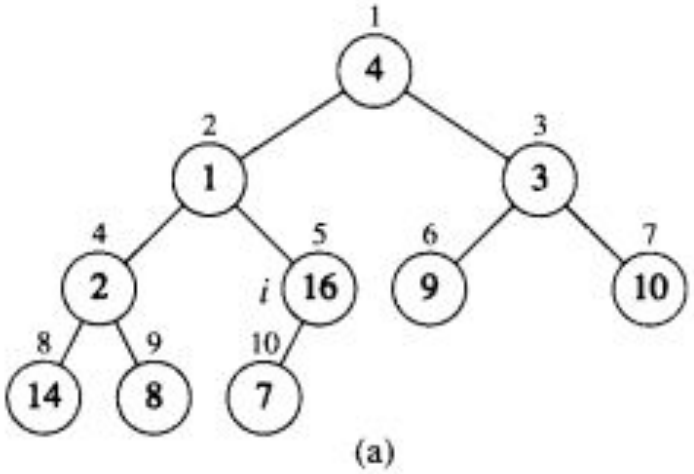


Heap Binária (build heap)

Constrói a heap a partir de um array

[4,1,3,2,16,9,10,14,8,7]

Simulação

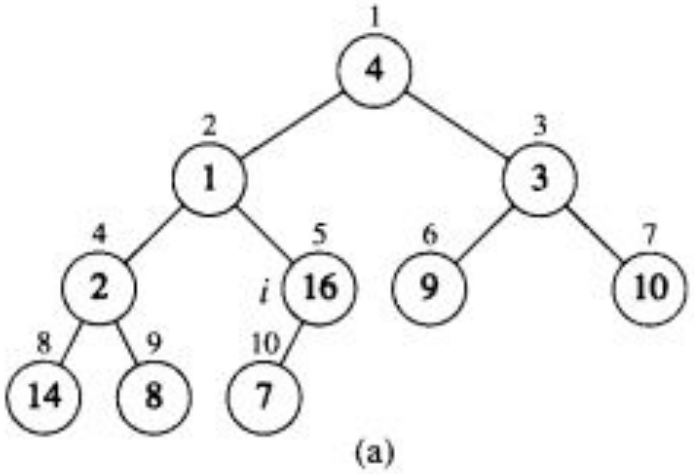


Heap Binária (build heap)

Constrói a heap a partir de um array

[4,1,3,2,16,9,10,14,8,7]

```
BUILD-HEAP(A)
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
        do HEAPIFY(A, i)
```



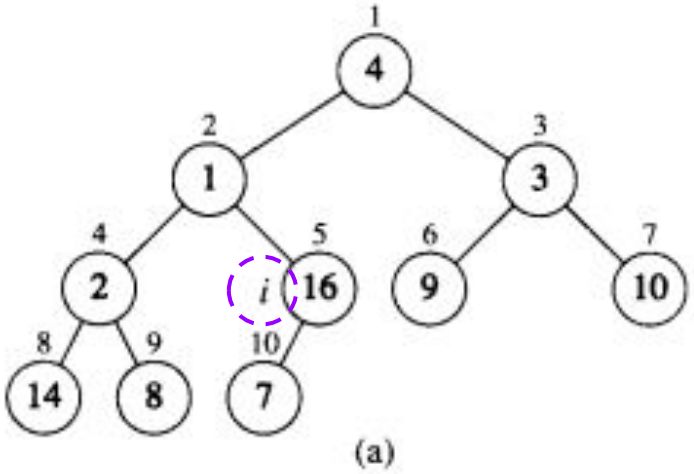
Heap Binária (build heap)

Constrói a heap a partir de um array

[4,1,3,2,16,9,10,14,8,7]

```
BUILD-HEAP(A)
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
        do HEAPIFY(A, i)
```

?

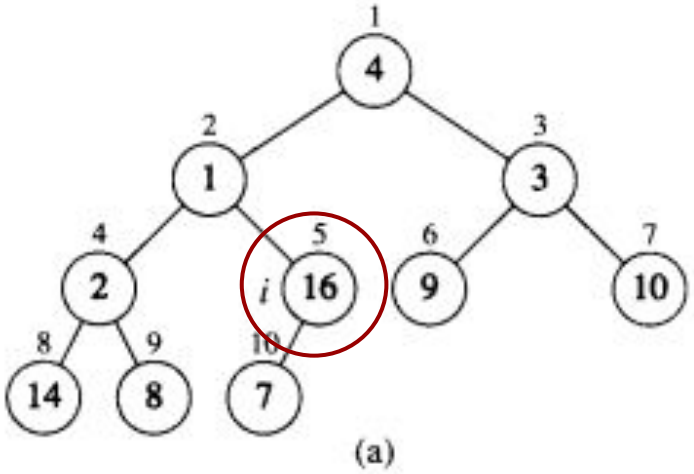


Heap Binária (build heap)

Constrói a heap a partir de um array

[4,1,3,2,16,9,10,14,8,7]

```
BUILD-HEAP(A)
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
        do HEAPIFY(A, i)
```

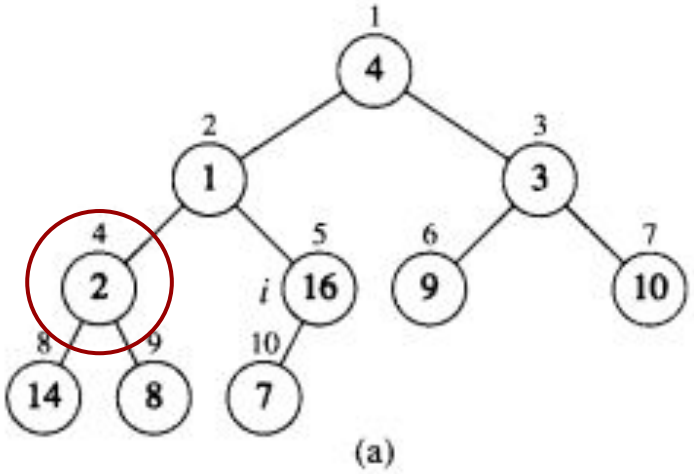


Heap Binária (build heap)

Constrói a heap a partir de um array

[4,1,3,2,16,9,10,14,8,7]

```
BUILD-HEAP(A)
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
        do HEAPIFY(A, i)
```

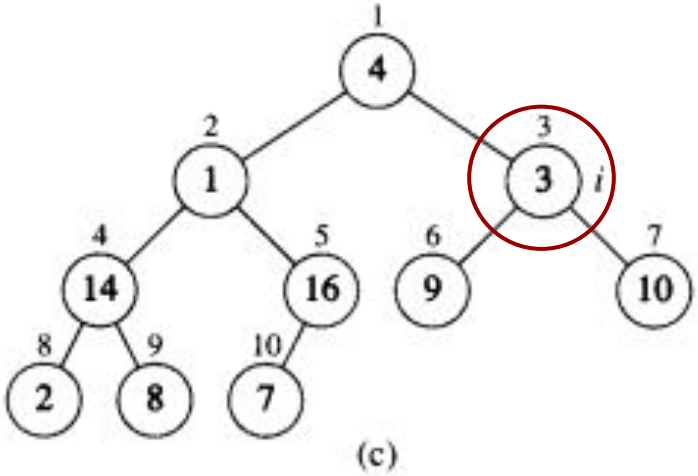


Heap Binária (build heap)

Constrói a heap a partir de um array

```
BUILD-HEAP(A)
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
        do HEAPIFY(A, i)
```

[4,1,3,14,16,9,10,2,8,7]

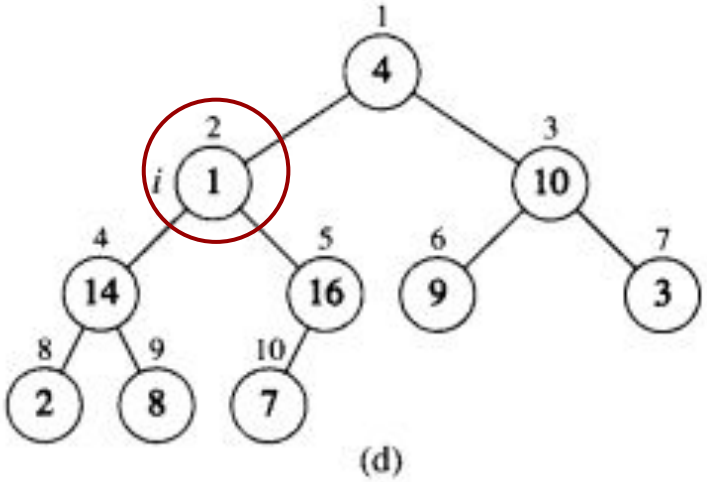


Heap Binária (build heap)

Constrói a heap a partir de um array

[4,1,10,14,16,9,3,2,8,7]

```
BUILD-HEAP(A)
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
        do HEAPIFY(A, i)
```

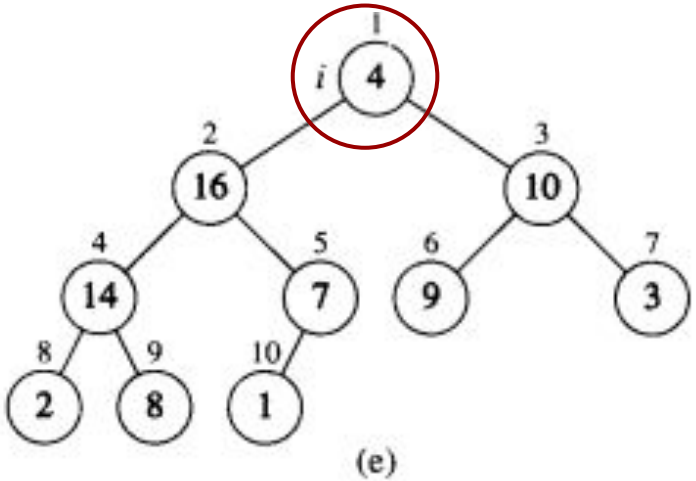


Heap Binária (build heap)

Constrói a heap a partir de um array

[4,16,10,14,7,9,3,2,8,1]

```
BUILD-HEAP(A)
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
        do HEAPIFY(A, i)
```

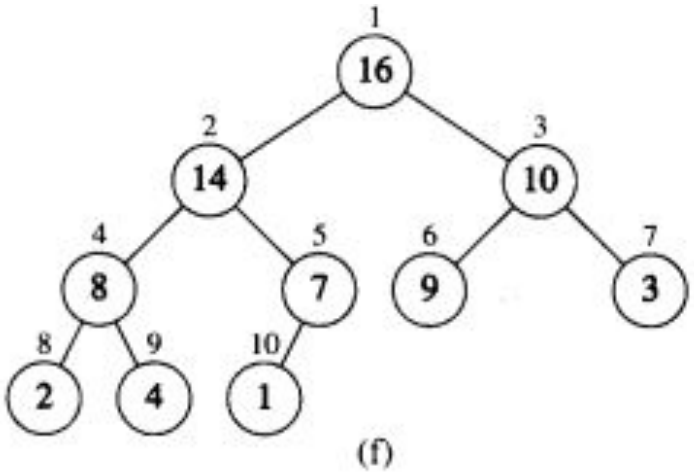


Heap Binária (build heap)

Constrói a heap a partir de um array

[16,14,10,8,7,9,3,2,4,1]

```
BUILD-HEAP(A)
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
        do HEAPIFY(A, i)
```



Heap Binária (build heap)

Qual a complexidade?

```
BUILD-HEAP(A)
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
        do HEAPIFY(A, i)
```

Heap Binária (build heap)

Qual a complexidade?

```
BUILD-HEAP(A)
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
    do HEAPIFY(A, i)
```

$O(n/2.\log n)$

Heap Binária (build heap)

Qual a complexidade?

```
BUILD-HEAP(A)
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
    do HEAPIFY(A, i)
```

$O(n/2.\log n)$

Análise mais detalhada mostra que é $\boldsymbol{\theta}$(n)

Varia de acordo com o nó (maioria tem altura pequena

Heap Binária (inserção)

Insere na heap e mantém o invariante

- Insere na última posição
- Sobe o elemento se necessário até manter o invariante

Simulação

Heap Binária (inserção)

```
HEAP-INSERT(A, key)
heap-size[A] ← heap-size[A] + 1
i ← heap-size[A]
while i > 1 and A[PARENT(i)] < key
      do A[i] ← A[PARENT(i)]
         i ← PARENT(i)
A[i] ← key
```

Qual a diferença básica com o Heapify?

Qual a complexidade?

Heap Binária (inserção)

```
HEAP-INSERT(A, key)
heap-size[A] ← heap-size[A] + 1
i ← heap-size[A]
while i > 1 and A[PARENT(i)] < key
      do A[i] ← A[PARENT(i)]
         i ← PARENT(i)
A[i] ← key
```

Qual a diferença básica com o Heapify?

Qual a complexidade?

$\theta(\log n)$

Heap Binária (remoção)

Remove da heap e mantém o invariante

- Remove **sempre** o elemento da raiz
- Move o último elemento da heap para a raiz
- Aplica heapify

Simulação

Heap Binária (remoção)

```
HEAP-EXTRACT-MAX(A)
if heap-size[A] < 1
    then error "heap underflow"
max ← A[1]
A[1] ← A[heap-size[A]]
heap-size[A] ← heap-size[A] − 1
HEAPIFY(A, 1)
return max
```

Qual a complexidade?

Heap Binária (remoção)

```
HEAP-EXTRACT-MAX(A)
if heap-size[A] < 1
    then error "heap underflow"
max ← A[1]
A[1] ← A[heap-size[A]]
heap-size[A] ← heap-size[A] − 1
HEAPIFY(A, 1)
return max
```

Qual a complexidade?

$\theta(\log n)$

Heap Binária (heapsort)

Ordena um conjunto de dados usando uma heap

- Troca a posição da raiz com o último elemento (raiz já vai para o final do array)
- Decrementa o tamanho da heap
- Aplica heapify na raiz

Simulação

Heap Binária (heapsort)

```
HEAPSORT(A)
BUILD-HEAP(A)
for i ← length[A] downto 2
    do exchange A[1] ↔ A[i]
       heap-size[A] ← heap-size[A] − 1
       HEAPIFY(A, 1)
```

Qual a complexidade?

Heap Binária (heapsort)

```
HEAPSORT(A)
BUILD-HEAP(A)
for i ← length[A] downto 2
    do exchange A[1] ↔ A[i]
       heap-size[A] ← heap-size[A] − 1
       HEAPIFY(A, 1)
```

Qual a complexidade?

θ(n log n)

Heap Binária (heapsort - abordagem com min-heap)

Ordena um conjunto de dados usando uma heap

- Insere todos os elementos do array na min-heap
- Faz n remoções da heap (n log n)

Heap Binária (aplicação - Priority Queue)

inserir, próximo, remover próximo

Heap Binária (aplicação - Priority Queue)

```
public class PriorityQueue<E>
extends AbstractQueue<E>
implements Serializable
```

An unbounded priority queue based on a priority heap.
a Comparator provided at queue construction time, dep
priority queue relying on natural ordering also does not
ClassCastException).

Min-heap

Comparator

Como transformar uma PriorityQueue em uma max-heap?

Heap Binária (aplicação - Priority Queue)

```
queue =
   new PriorityQueue<Integer>((o1,o2) -> o1 - o2);

            o1 = 5, o2 = 3 ⇒  o1 - o2 = 2 (>0)

queue =
   new PriorityQueue<Integer>((o1,o2) -> o2 - o1);

            o1 = 5, o2 = 3 ⇒  o2 - o1 = -2 (<0)
```

Heap Binária (aplicação - Priority Queue)

```
class NormalComparator<T extends
Comparable<T>> implements Comparator<T>{
   public boolean compare(T o1, T o2){
    return o1.compareTo(o2);
   }
}
```

```
class InvertedComparator<T extends
Comparable<T>> implements Comparator<T>{
   public boolean compare(T o1, T o2){
    return o2.comparaTo(o1);
   }
}
```

```
queue =
   new PriorityQueue<Integer>(new NormalComparator<Integer>());

queue =
   new PriorityQueue<Integer>(new InvertedComparator<Integer>());
```

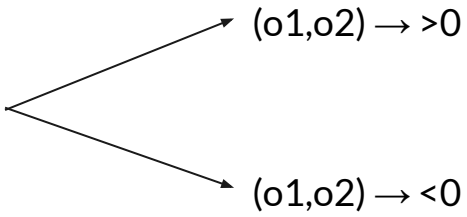
Heap Binária (aplicação - Pilha e Fila)

É possível implementar pilha e fila com PriorityQueue? Como?

Heap Binária (aplicação - Pilha e Fila)

É possível implementar pilha e fila com PriorityQueue? Como?

PriorityQueue
- comparator



Heap Binária (aplicação - Pilha e Fila)

É possível implementar pilha e fila com PriorityQueue? Como?

PriorityQueue
- comparator

(o1,o2) → >0

tempo (timestamp)

(o1,o2) → <0

Heap Binária (aplicação - Pilha e Fila)

É possível implementar pilha e fila com PriorityQueue? Como?

PriorityQueue
- comparator

(o1,o2) → o1.timeStamp - o2.timeStamp

... o1 .... o2 ....

(o1,o2) → o2.timestamp - o1.timeStamp

Heap Binária
Implementação

Heap Binária - Implementação

Elementos comparáveis

Comparator interno (mais flexibilidade para a estrutura)

Heap Binária - Implementação

```
public interface Heap<T extends Comparable<T>> {
  public void insert(T element);
  public T extractRootElement();
  public T rootElement();
  public T[] heapsort(T[] array);
  public void buidHeap(T[] array);
  public T[] toArray();
  public int size();
}
```

Heap Binária - Implementação

```
public class HeapImpl<T extends Comparable<T>> implements Heap<T> {
    protected T[] heap;
    protected int index = -1;
    protected Comparator<T> comparator;
    private static final int INITIAL_SIZE = 20;
    private static final int INCREASING_FACTOR = 10;

    public HeapImpl(Comparator<T> comparator) {
        this.heap = (T[]) (new Comparable[INITIAL_SIZE]);
        this.comparator = comparator;
      }
}
```